

Part 8

Graphical Processing Units

8.1 Why is efficient threading tricky on CPUs ?

So far we have almost exclusively focused on programming central processing units (CPU) in serial with C, then with OpenMP annotated C for multithreading to exploit multiple CPU cores, and subsequently distributed computing to run on multiple CPUs by calling MPI from C. In some cases we have only seen modest performance gains from multithreading. Furthermore, we have discussed the difficulty of persuading the compiler to generate vector instructions that instruct the CPU SIMD units to perform multiple arithmetic operations simultaneously per core. The compiler is cautious to introduce these instructions because of the requirement that it can guarantee all data inputs and outputs to the vector instructions need to be aligned to appropriate memory page boundaries and that the full vector width can be exploited in the calculation.

I (Prof. Tim Warburton) have personally observed the progression in vector instruction sets for Intel CPUs. Recalling the [MMX](#) vector extensions released in 1997, the Streaming SIMD Extensions ([SSE](#)) released in 1999, the enlarged instructions set extensions [SSE2](#), [SSE3](#), [SSE4](#), followed by the Advanced Vector Extension [AVX](#), the sequel [AVX2](#), and the even wider SIMD follow on [AVX-512](#). Each of these vector instruction extensions includes specialized functions for SIMD instructions of hardware specific vector operations. From a practical point of view it is not an attractive prospect to write code that targets the contemporary vector instruction knowing full well that it will need to be updated for the next CPU SIMD instruction set¹. It is possible to write code where the vector operations are isolated to small parts of the code, but the requirements of vector operations enforce global requirements on data alignment.

Many of the problems with CPU vector SIMD units boil down to their being retrofitted into the well established x86 core architecture. An optimizing compiler has to decide whether it is possible to engage the SIMD processor for a set of operations, for instance bulk processing arithmetic iterations from chunks of loop iterations. The compiler has to verify that all data is properly aligned, that there is no loop carried dependency, and apply heuristic models to estimate whether the vectorized operation would be faster than just using sequential operations. Most recently it has come to light that some late generation Intel CPUs throttle clock frequency when using the wide AVX-512 operations [reducing the performance below expectations](#). In summary: the vector extension sets for Intel CPUs are a moving

¹There is some useful information [here](#) about how to target specific families of CPU processors.

target; it is difficult to persuade CPU compilers to automatically generate code for the SIMD units; even if the compiler can be persuaded to vectorize code the CPU may reduce the CPU clock frequency.

8.2 Graphics processing units as an alternative to CPUs

The CPU is fortunately not the only game² in town for performing calculations on a computer. The popularity of home computers, video games, game consoles, and PC gaming in the 80s and 90s drove the parallel³ development of graphics processing units (GPUs). Typical early GPUs were highly specialized processors that were designed to efficiently render polygonal 3D scenes into a display buffer at acceptably playable resolution and framerate. Manufacturers including ATI (now AMD), NVIDIA, SGI, Sony and others produced GPUs starting in the early 1990s. Their GPUs were used to accelerate the rendering of graphics in a wide range of settings including professional workstations (SGI), PC desktops (ATI, NVIDIA, 3dfx), and game consoles (Sega, NVIDIA, AMD). More recently laptops and cell phones have crucially depended on GPUs to deliver high resolution (4K) displays for relatively low energy overhead given that these are battery powered devices.

The use cases envisioned for GPUs initially was very different to CPUs. In the 1990s the former only had to excel at rendering graphics in a timely fashion whereas the latter was expected to be capable of general purpose computing tasks encompassing business applications including payroll, word processing, spreadsheets, CAD, CAE, databases, and countless other user facing applications. The GPU had to mediate between the CPU and the display, but the CPU had to also coordinate physical storage (CD drive, hard drive, tape drive), network, user interface (mouse, keyboard, trackball...), and a myriad of attached devices.

The specialized tasks of early GPUs enabled the chip designers to build computational cores with lower complexity than general purpose CPU cores. Furthermore since the tasks required to color pixels on a display are the same for each pixel it was possible to employ ab initio hierarchical parallelism with the capability to concurrently execute a large number of hardware threads. Each product in the modern NVIDIA GPU lineup consists of multiple compute cores⁴. Each compute core is equipped with one or more vector SIMD arithmetic logic units, i.e. floating point units.

The progression of GPUs since their inception in the 1990s has been remarkable. For instance the number of transistors used in GPUs has followed a Moore's Law like trend with exponential growth maintained for 20 years as shown in Figure 8.1, only showing signs of slow down in the 2013-2015 where both NVIDIA and AMD got "stuck" at the 28nm manufacturing process due to the technical challenges in transitioning to 14nm. From my personal experience although it might appear that the GPUs did not progress in manufacturing process scale, the manufacturers actually used node improvements in the core microarchitecture and the GPUs did improve in performance during this time.

²See what we did there?

³Unavoidable, really.

⁴NVIDIA calls each GPU core a Streaming Multiprocessor (SM).

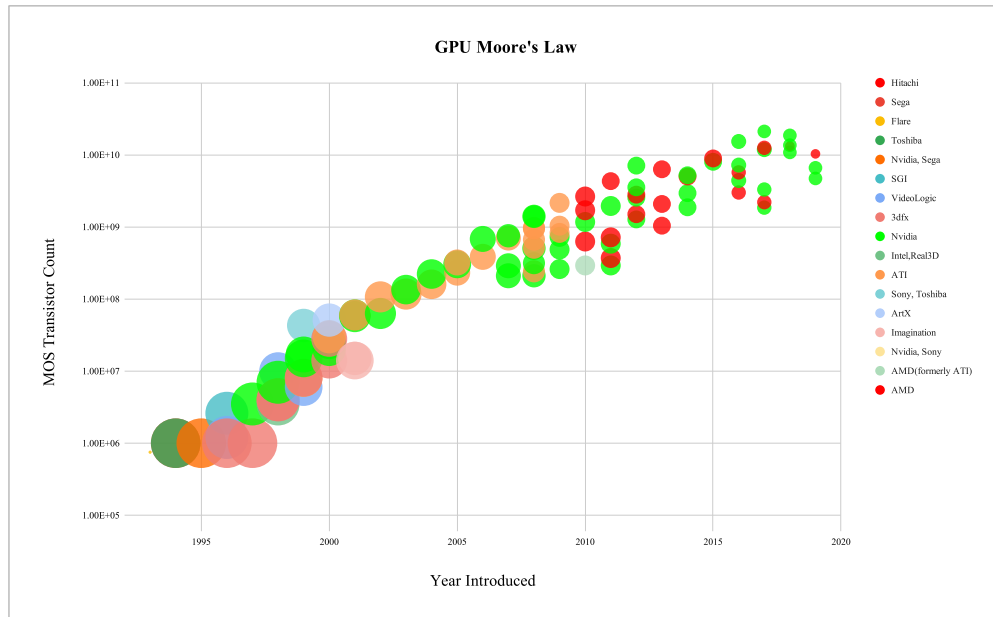


Figure 8.1: Moore's law for GPUs: data obtained from [wiki page on transistor counts](#). Nearly thirty years of data show continued exponential increase in transistor counts, with some noticeable dips in the trend. For instance 2013-2015 there were delays in transitioning between 28nm and 14/16nm manufacturing processes. Scatter symbols are scaled by manufacturing process size with the latest AMD GPUs manufactured at 7nm.

Acknowledgement of source for CPU/GPU performance trend data and figures

In the following we include figures sourced from [Karl Rupp's webpage](#) and [his GitHub repository](#). His resources include an excellent write up on the trends in GPUs that I strongly recommend you review his commentary. All figures here were regenerated as of 10/06/2019 and are subject to the license agreement [here](#).

Shrinking transistors over time has allowed many more floating point units to be packaged into each GPU. This has seen an increase in peak double precision from $\mathcal{O}(100)$ GFLOPS/s in the mid 2000s to $\mathcal{O}(10,000)$ GFLOPS/s in 2018 as shown in Figure 8.2. In that Figure it is also clear that the peak performance has plateaued between shrinks of the manufacturing process for transistors, but my experience is that gap between achievable and peak performance narrowing during the plateaus. We also note that chart shows a considerable gap between contemporary CPUs and GPUs.

One thing to keep in mind about the increase in peak performance in GPU double precision performance is that it could be undone if the energy requirement per FLOP increases. Fortunately in Figure 8.3 we show that the number of GFLOPS/per Watt by and large has increased over time. Also, there is a considerable energy bonus from using GPUs comparing to CPUs given the gap between their respective GFLOPS/per Watt plot lines.

At this point one might ask: assuming that the GPU architectures are eminently scalable⁵ what limits

⁵Assuming that GPU architectures are indefinitely scalable sidesteps several issues that get in the way. For instance although GPUs are typically run at lower clock frequencies than the CPU cousins, the GPUs still require substantial effort to cool and this relies in turn on continuous improvement on processor cooling technology. A second major issue that we will outline in more detail later is that recent GPUs use an L2 cache shared amongst all 80 cores [1]. History

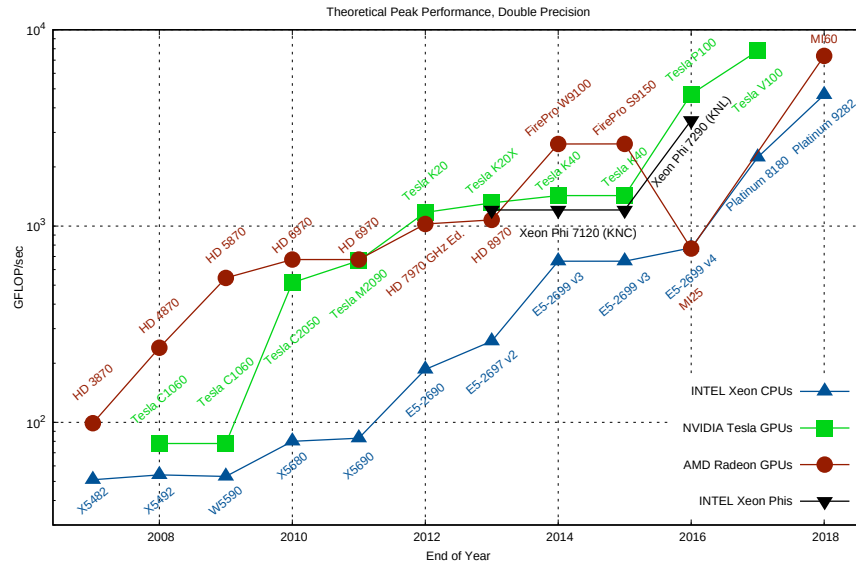


Figure 8.2: Trend in theoretical peak GPU double precision floating point performance over time. Image generated using Karl Rupp's scripts and data sourced from [his GitHub repository](#) subject to [this license](#).

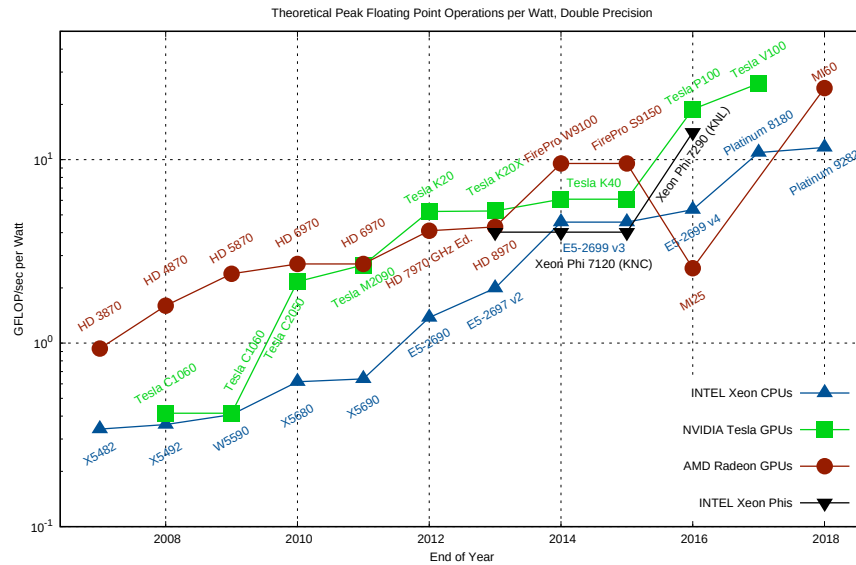


Figure 8.3: Trend in theoretical peak GPU double precision floating point performance normalized by energy consumption over time. Image generated using Karl Rupp's scripts and data sourced from [his GitHub repository](#) subject to [this license](#).

the number of cores, the number of floating point units, and thus the overall peak floating point performance? In Figure 8.4 we show the maximum power consumption (also referred to as “Thermal Design Power” typically shortened to TDP) for several generations of CPUs and GPUs. For several generations the TDP for GPUs was limited to 250 Watts primarily because the GPUs were connected to the host computer via the PCI-E card interface. The PCI-E standard limits the power consumed by attached devices to 250 Watts, hence the plateau evident in Figure 8.4. You may also notice the uptick in GPU TDP in 2017 when some NVIDIA Volta V100 GPUs were released with a proprietary hardware interface to the host computer that did not share the same TDP limit as PCI-E. At the same time you will notice that Intel started to release server CPUs with large increases in their TDP, in part to produce CPUs that are closer in peak floating point performance to the top rated GPUs from AMD and NVIDIA.

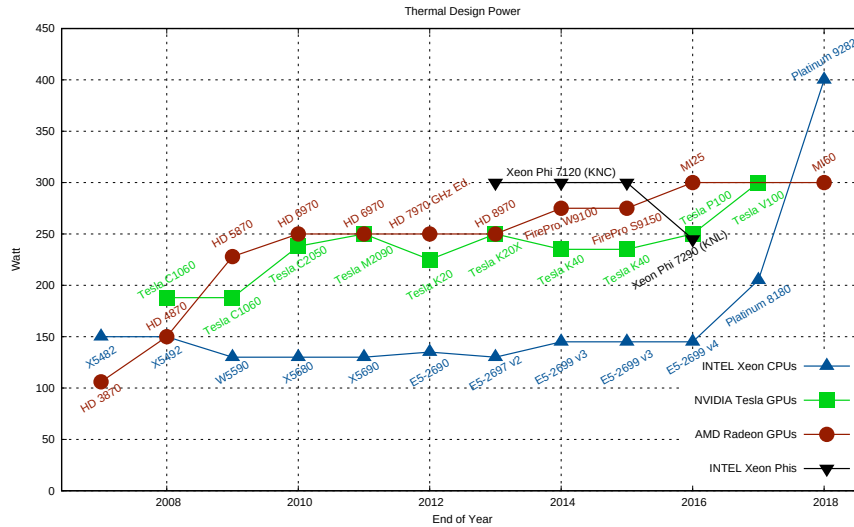


Figure 8.4: Trend GPU thermal design power maximums over time. Image generated using Karl Rupp’s scripts and data sourced from [his GitHub repository](#) subject to [this license](#).

8.3 Example GPU: the NVIDIA Volta V100

To make this discussion of GPU architectures concrete we consider the NVIDIA Volta V100 GPU. It has 80 compute cores as illustrated in Figure 8.5.

Each core is equipped with 32 double precision floating point units per compute core as illustrated in Figure 8.6. Thus the V100 has 2,560 double precision floating point units. Each of these floating point units can execute a Fused Multiply ADD (FMADD) per clock cycle. Furthermore, the clock frequency of the Volta core is dynamically clocked up to 1.53GHz. Thus the V100 can theoretically perform

$$|\# \text{ CORES}| \times |\# \text{ SIMD}| \times |\# \text{ OPS/FMADD}| \times |\text{FREQUENCY}| = 80 \times 32 \times 2 \times 1.53 \text{ GHz} \approx 7.8 \text{ TFLOPS/s.}$$

Currently this peak performance values are about 30 TFLOPS/s in NVIDIA H200 high-end GPUs.

As an aside the observant reader will notice in Figure 8.6 that each V100 SM also includes compute units labelled as “Tensor Core”. These are highly specialized hardware for [computing half preci-](#)

suggests that this arrangement will not scale to a large number of cores given the challenges of coordinating requests from a large number of cores to a shared memory cache system.



Figure 8.5: Block diagram for the NVIDIA Volta V100 GPU. Image source: [NVIDIA Volta V100 White Paper](#).

sion (16 bit) tensor operations targeting machine learning applications. Some researchers have coaxed these low precision tensor cores into performing more general purpose calculations using iterative refinement algorithms [2, 3, 4, 5, 6]. By and large however many GPU enabled applications do not exploit the extra performance bumps available from the tensor cores because of their limitation to processing floating point variables with half precision.

The 32-wide SIMD units are central to the design of the GPU cores and achieve high throughput for good reason: whereas vectorization was a late addition to CPU cores it was baked into the GPU compute cores from the start. In fact until recently all operations on NVIDIA GPUs are processed as full 32-wide vector operations. Thus forcing a GPU core to perform a scalar operation is the exception rather than the norm. In fact it can be easier to write SIMD parallelized code for the GPU than for the CPU!

8.4 GPU memory system

So far we have focused on the abundance of floating point units in modern GPUs, in particular the over provisioned NVIDIA Volta GPU. You might ask, why is it “over provisioned” ? The reality is that the GPU cores are routinely idled during calculations because there are so many floating point units that the GPU device system memory is unable to provide data at a fast enough rate to keep them occupied, i.e. they consume available data too fast !

To understand the imbalance between the GPU floating performance and GPU device memory capability, consider the block diagram of the NVIDIA Volta V100 shown in Figure 8.7. A typical Volta class GPU has up to 80 compute cores. Each equipped with just 256KB of register memory and 128KB of unified L1/shared memory. The registers, cache, and shared scratch pad have to be shared amongst 32 double precision floating point units. That means that all the data that threads need to complete their task must reside in this fast memory, otherwise they must request data load/stores from device



Figure 8.6: Core diagram for the NVIDIA Volta V100 GPU Streaming Multiprocessor. Image source: [NVIDIA Volta V100 White Paper](#).

memory. This has significant ramifications for the type of computational tasks suitable for a GPU thread.

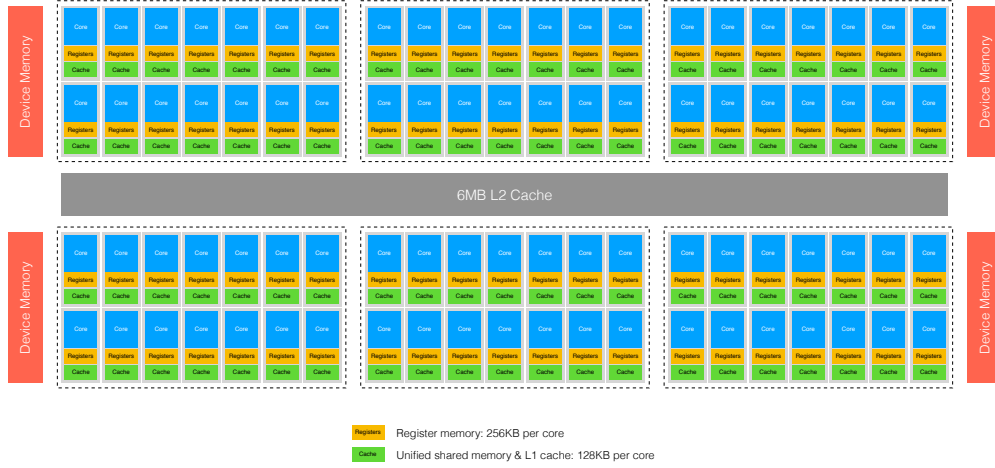


Figure 8.7: Block diagram of memory hierarchy of the 84 core full NVIDIA GV100 GPU.

In Figure 8.8 we show the ratio of flops per byte required to achieve peak performance when data is streamed from off-chip memory. We will refer to this ratio as the peak arithmetic intensity.

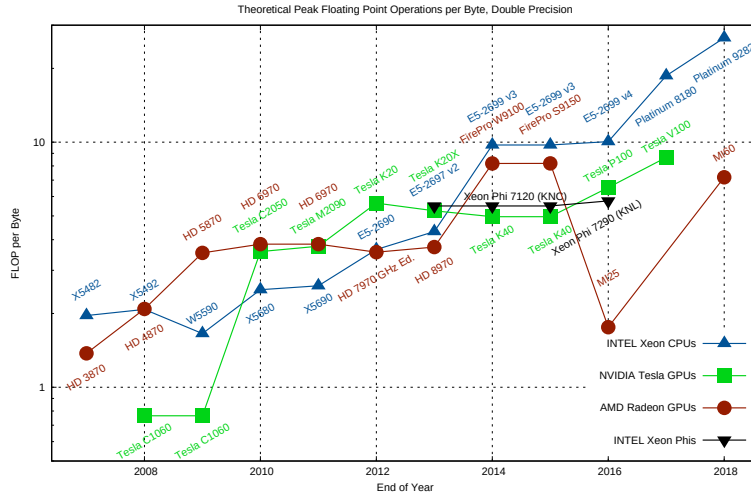


Figure 8.8: Trend in the number of double precision floating point operations required per byte of data to achieve peak performance. Image generated using Karl Rupp’s scripts and data sourced from [his GitHub repository](#) subject to [this license](#).

Recent GPUs require a peak arithmetic intensity close to 10 double precision floating point operations per byte loaded or stored from off-chip memory. Very few computational tasks require an arithmetic intensity of 10. Some examples assuming double precision arithmetic are given in Table 8.1.

In these examples the arithmetic intensity is well below the optimal value of ten. Revisiting Figure 8.8 in light of these practical examples and we should be rather disappointed that even a fully optimized implementation of such operations will only take advantage of a small fraction of the floating point capability of the V100. A computation with arithmetic intensity of approximately 0.1 can only extract

Operation	Math	Reads	Writes	Arithmetic intensity
Add	$z_i \leftarrow x_i + y_i$	$2N$	N	$1/(3 \times 8) \approx 0.04$
Axpy	$y_i \leftarrow \alpha x_i + y_i$	$2N$	N	$2/(3 \times 8) \approx 0.08$
Dot product	$a \leftarrow \sum_i x_i y_i$	$2N$	1	$2/(2 \times 8) \approx 0.12$
Self dot product	$a \leftarrow \sum_i x_i^2$	N	1	$2/(1 \times 8) \approx 0.25$
Matrix-vector product	$y_i \leftarrow \alpha \sum_j A_{ij} x_j + \beta y_i$	$N^2 + 2N$	N	$(2N^2 + 3N)/((N^2 + 3N) \times 8) \approx 0.25$

Table 8.1: Estimates of arithmetic intensity (floating point operations per byte of data moved) for a set of standard operations using double precision arithmetic. Reads and writes are given in terms of minimum doubles that can be read and written. Here all vectors are assumed to have N entries, all matrices are assumed to be of size $N \times N$, and a, α, β are all scalar variables.

1% of a GPU that requires an arithmetic intensity of about 10 to achieve peak streaming calculation performance!

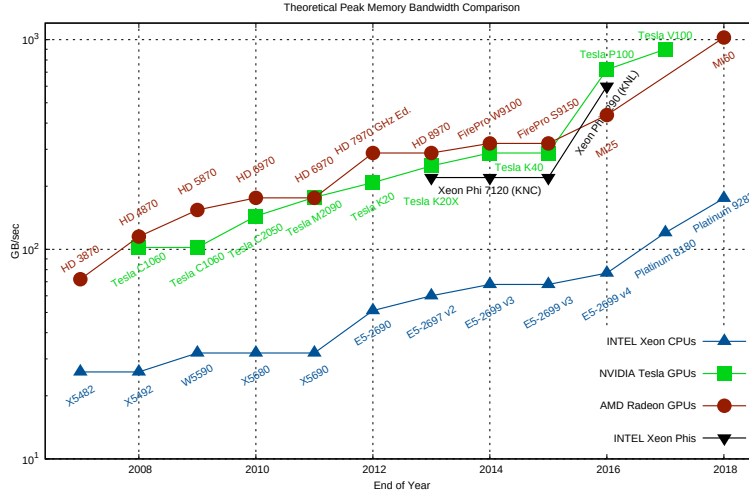


Figure 8.9: Trend in maximum theoretical peak memory bandwidth for CPUs and GPUs over time. Image generated using Karl Rupp’s scripts and data sourced from [his GitHub repository](#) subject to [this license](#).

The inevitably disappointing floating point performance of these ubiquitous vector operations due to their low intrinsic arithmetic intensity is not game over. In practice when presented with operations like this that are “memory bound” calculations we pivot. Instead of maximizing floating point performance we acknowledge that performance is limited by how fast data can be streamed through the GPU memory system and we try to maximize data throughput. In Figure 8.9 we show trends over time for the throughput of GPU memory systems compared to CPU memory systems. We see that there is a clear and unambiguous gap between GPU and CPU memory throughput. This is actually the secret sauce of why GPUs are gaining market share from CPUs as the computational workhorse in data centers, clusters, data processing warehouses, and super computers. The GPUs use faster memory with wider data buses than CPUs do, a trend that goes back to the earliest days when GPUs were used exclusively for rendering graphics. That task is heavily memory bound and performance depends almost entirely on the data throughput capability of the GPU’s attached memory system.

8.5 Ongoing developments in GPU core architecture

We have seen already that the transistor counts for successive generations of GPUs has followed a version of Moore's Law. You can review the lineage of NVIDIA GPUs that precede the NVIDIA V100 and H200 cards at this [wiki page](#). The first Tesla branded GPU was the c780. In Table 8.2 we compare with NVIDIA V100 and the recent H200 GPU to illustrate the astonishing improvements that happened between 2007, 2017 and 2024.

Feature	Tesla c780	Tesla V100	Hopper 200 NVL
Year introduced	2007	2017	2024
Clock frequency	600 MHz	1530 MHz	3.5 MHz
Memory bandwidth	78 GB/s	900 GB/s	4800 GB/s
Peak FP32 throughput	346 GFLOPS/s	14,000 GFLOPS/s	60, 000 GFLOPS/s
Peak FP64 throughput	–	7,000 GFLOPS/s	30,000 GFLOPS/s
Core count	16	80	132
FP32 FPU per core	8	64	256
FP64 FPU per core	-	32	128

Table 8.2: Comparison of the earliest NVIDIA server GPU, the Tesla c870 with Tesla V100 and the modern H200 NVL . Specs sourced from [techpowerup GPU database](#) and [wiki list of NVIDIA Tesla GPUs](#).

Note: in a decade the floating point throughput has increased by a factor of 40 but the memory bandwidth only increased by a factor of 11. This tracks with the increase we see in the arithmetic intensity trend Figure 8.8. However, this is a super worrying issue when we contemplate future GPUs. The problem is that the memory throughput is increasing more slowly each generation than the floating point performance is increasing. We effectively are wasting silicon or just idling compute cores while they wait for data to arrive.

The manufacturers have responded to the memory bandwidth gap increasing over time by resisting the urge to maximize the number of vanilla compute cores per GPU. For instance NVIDIA's latest consumer GPUs have added specialized compute units to each compute core. There are two types of specialized co-co-processors: the ray tracing core in the NVIDIA RTX GPUs perform custom operations to accelerate scene rendering and the tensor cores in the NVIDIA Volta class GPUs and later generations perform specialized low precision matrix multiplication operations. Each of these two new types of core perform a large number of operations per input value, and furthermore open up GPU computing to new applications. Ray tracing before the advent of customized cores was not feasible for realtime rendering in games, but now Battlefield 4 and other triple A titles can run with raytracing enabled for enhanced visuals but with a reduced display framerate (to the dismay of serious gamers). The tensor cores are being used for training neural networks on cloud instances to enable deep learning with very wide neural networks.

A cynic might observe that NVIDIA has to find new application areas to grow its potential market because the trend of everfaster and faster GPUs has stalled because of the memory bandwidth crisis.

So what's next ?: the cost of designing and manufacturing high end processors has significantly narrowed the number of chip fabrication facilities worldwide. At the time of writing the main manufacturer of the latest 7nm based processors is the Taiwan Semiconductor Manufacturing Company (TSMC). This is used by the latest AMD GPUs and unfortunately also the latest iPhone 11 phones. The demand for the iPhone is strong and consequently the new generations of AMD GPUs will be delayed. The challenges of producing processors at 7nm means that there is now a supply problem and both prices and delivery times will increase. You can expect the price of your iPhone and GPUs

to increase accordingly. Overall, the lack of competition in manufacturing will impact cost. But the major existential threat to next generation GPUs is the difficulty and cost of manufacturing them.

Intel is addressing the production issues by switching from building a processor as a monolithic chip to building instead a processor as a collection of chiplets. The idea is that a chip will be constructed from a set of smaller building blocks that will cost less in design and have higher yield in production since more chiplets can be constructed from a single wafer and the cost of a dud chiplet is lower than the cost of a dud monolithic chip.

Looking further ahead we can speculate that chips will be constructed from stacked chiplets to reduce the sprawl inherent in 2D chip design. This will need to be accompanied with memory that is collocated with the chiplets. Fingers crossed that this technology becomes viable in the near future.

8.6 GPU programming models

GPUs have also transitioned to general purpose computing devices. At inception GPUs were viewed as niche products suitable for gaming and rendering on graphics workstations. But since the mid 2000s they have been adapted to more general purpose computing applications. This coincides with the so-called end of Dennard scaling around 2005 when increasing CPU processor clock frequencies hit the thermal wall. Suddenly there was an opportunity for competition in the processor space since applications were no longer sped up just by buying the latest Intel or AMD CPU. Furthermore, software developers were having limited success squeezing good performance out of the new multi-core CPUs.

In 2007-2008 NVIDIA released GPUs with new compute capabilities and a new programming model called the Compute Unified Device Architecture (CUDA) to exploit the new hardware features. Researchers found that if they ported their applications to CUDA that they could get remarkable speed ups. In part this was due to higher peak performance and higher memory throughput, but the performance bumps in many instances could be attributed to the key feature that the GPU is *ab initio* parallel in core and vectorization. Whereas developers struggled to persuade compilers to generate SIMD parallel code for CPUs, they couldn't help but generate SIMD parallel code when writing CUDA code at least if they paid a modicum of attention to the GPU architecture and CUDA language features. In the following lectures we will describe the CUDA programming language and add more detail to inform your mental model of a modern GPU architecture.

8.7 GPU computing references

We list some books that are useful sources of introduction material. The thumbs up are my personal assessment of the texts.

Presentations about GPUs:

CUDA books:

1. [7]: from the blurb “*CUDA now brings this valuable resource to programmers working on applications in other domains, including science, engineering, and finance. No knowledge of graph-*

ics programming is required; just the ability to program in a modestly extended version of C. *CUDA by Example*, written by two senior members of the CUDA software platform team, shows programmers how to employ this new technology. The authors introduce each area of CUDA development through working examples. After a concise introduction to the CUDA platform and architecture, as well as a quick-start guide to CUDA C, the book details the techniques and trade-offs associated with each key CUDA feature.” (👍👍)

2. [8]: from the blurb “*This book is intended to serve as a practical guide for the development and implementation of numerical algorithms on Graphics Processing Units (GPUs). The book assumes that the reader is familiar with the mathematical context and has a good working knowledge of GPU.*” (👍👍 and available via [SpringerLink](#))
3. [9]: from the blurb “*In addition, the book provides many case studies, which are used to explain and reinforce important GPU concepts like CUDA threads, the GPU memory hierarchy, and scalability across multiple GPUs including an MPI example* ” (👍)
4. [10]: from the blurb “*Designed for professionals across multiple industrial sectors, Professional CUDA C Programming presents CUDA – a parallel computing platform and programming model designed to ease the development of GPU programming – fundamentals in an easy-to-follow format, and teaches readers how to think in parallel and implement parallel algorithms on GPUs*” (👍)
5. [11]: from the blurb “*... shows both student and professional alike the basic concepts of parallel programming and GPU architecture, exploring, in detail, various techniques for constructing parallel programs.*” (👍👍👍)

OpenCL books:

1. [12]: from the blurb “ *... teaches OpenCL and parallel programming for complex systems that may include a variety of device architectures: multi-core CPUs, GPUs, and fully-integrated Accelerated Processing Units (APUs) such as AMD Fusion technology. It is the first textbook that presents OpenCL programming appropriate for the classroom and is intended to support a parallel programming course.* ” (👍)

Graphics processing units (GPUs) are ubiquitous. Pay attention for one day and see how many devices you interact with that contain some form of GPU. Some obvious electronic devices that you use frequently likely include a GPU: cell phone, tablet, laptop, desktop PC, workstation, cluster, cloud computing server, and game console. However, GPUs are also used in other consumer devices including: modern automobile (in particular Teslas - driving the console display or even driving the car), personal digital assistant (locally for the display and/or parsing speech input), smartwatch, tv streaming device (for decoding), some smart tvs,

If you drove your Tesla (or more to the point it drove itself) to pick up some avocado toast, used a self checkout, paid with bitcoin, swiped your cell phone for coupons on that sweet avocado and artisan bread, did some work on your laptop, while streaming some Netflix, and ran some data analysis in the cloud today while demanding things from Alexa/Siri/Cortana/Google and taking a call on your smartwatch you have interacted with a surprisingly large number of GPUs without even consciously doing so. Sit back and relax on the couch and spend some well deserved time playing with your game console, that was a busy day.

In many cases these consumer devices include so-called “system-on-chips” which package low power CPU cores, GPU cores, memory, as well as video decoders etc. The GPU cores are managing much

of the large data tasks associated with displaying video, decoding/encoding audio, and painting fancy graphics. They are specifically designed because they require less energy than CPU cores for these tasks in particular for applications where power usage is important, for example any of these devices that is battery powered like the phone, tablet, or smartwatch.

GPUs are an example of an [ASIC](#) (Application Specific Integrated Circuits) processor initially designed to accelerate rendering of 3D graphics for games. However, they were co-opted in the mid-2000s for more general purpose tasks as they were engineered to be less application specific and less focused purely on gaming applications. The field of general purposes GPU (GPGPU) computing coincided with the release of NVIDIA's Compute Unified Device Architecture ([CUDA](#)) programming language and API in 2007. NVIDIA positioned CUDA to extend the range of applications that GPUs could be used for. My personal interest in GPGPU computing was sparked by the impressive performance gains early adopters reported for their CUDA implementations. CUDA was released at about the time that CPUs were no longer getting clock bumps and were transitioning to multi-core architectures with some teething problems that meant getting good multi-threaded performance was not easy.

8.8 GPU overview

Before we dive into the specifics of the CUDA programming language that we will use to program NVIDIA GPUs we need to know something about GPU architectures. In these notes we will focus on discrete GPUs, that is GPUs which are distinct and separate from the CPU ⁶.

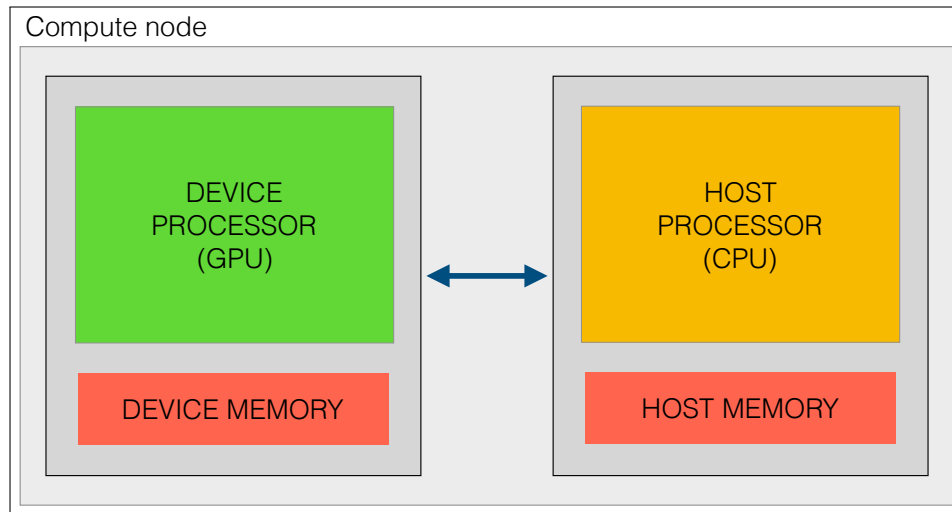


Figure 8.10: Much simplified block diagram of compute node HOST (CPU) with one DEVICE (GPU). The double arrow line represents the interface between DEVICE and HOST, for example PCI-E or NVLink.

In Figure 8.10 we show a greatly simplified block diagram representing the arrangement of a CPU and GPU. We will use the following terminology when talking about GPU computing:

HOST: the HOST is the main computer that a GPU is attached to. For instance if a desktop contains

⁶There are also integrated GPUs, or iGPUs, that combine CPU and GPU cores. Once you know how to program discrete GPUs with CUDA then it is a short hop to programming iGPUs usually via the Open Compute Language (OpenCL) that was in turn based on CUDA but is able to be used on a wider range of CPUs and GPUs from multiple manufacturers.

a motherboard with CPU and GPU, then the motherboard and CPU play HOST to the GPU.

DEVICE: the DEVICE is the GPU which includes the GPU processor equipped with its own GPU memory.

Notes #1: the HOST processor cannot directly access the DEVICE memory. Data must be transferred over the HOST<>DEVICE interface.

Notes #2: the DEVICE processor cannot directly access the HOST memory. Data must be transferred over the HOST<>DEVICE interface.

Notes #3: the data transfer rate between HOST and DEVICE is typically an order of magnitude slower than the rate between the DEVICE and its DEVICE memory. To get the best performance out of a heterogeneous compute node with both CPU and GPU it is advisable to keep the amount of data movement between the HOST and DEVICE to a minimum. Thus a typical calculation involves initializing data on the HOST, copying it to the DEVICE memory, doing as much work on the DEVICE as possible, then finally copying the result back to the HOST.

8.9 CUDA hello world

The standard CUDA “hello world” program for GPUs is a little different to the CPU hello world program. It is customary for the former to do the following

1. Allocate array on HOST.
2. Allocate array on DEVICE.
3. Launch DEVICE compute kernel to fill the DEVICE array with values.
4. Copy data from DEVICE array to HOST array.
5. Print out values from HOST array.

8.9.1 CUDA hello world: HOST code

The following CUDA code implements the HOST side operations, the DEVICE side kernel implementation follows later in these notes.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "cuda.h"
4
5  int main(int argc, char **argv){
6      // 1. allocate HOST array
7      int N = 1024;
8      int *h_a = (int*) calloc(N, sizeof(int));
9
10     // 2. allocate DEVICE array
```

```

11  int *c_a;
12  cudaMalloc(&c_a, N*sizeof(int));
13
14  // 3. launch DEVICE fill kernel
15  int T = 256; // number of threads per thread block
16  dim3 G( (N+T-1)/T ); // number of thread blocks to use
17  dim3 B(T);
18
19  int val = 999; // value to fill DEVICE array with
20  fillKernel <<< G,B >>> (N, val, c_a);
21
22  // 4. copy data from DEVICE array to HOST array
23  cudaMemcpy(h_a, c_a, N*sizeof(int), cudaMemcpyDeviceToHost);
24
25  // 5. print out values on HOST
26  for(int n=0;n<N;++n) printf("h_a[%d] = %d\n", n, h_a[n]);
27
28  // 6. free arrays
29  cudaFree(c_a);
30  free(h_a);
31  return 0;
32 }

```

You should notice that some of this code is nonstandard C. In fact the CUDA language contains extensions to standard C tailored for GPU specific operations. The following notes call out some of the special operations and syntax in the above CUDA code.

Note #1: we must include the CUDA header file `cuda.h` since it includes the prototypes for all CUDA API functions.

Note #2: all CUDA functions⁷ start with “cuda” and follow standard camel case notation.

Note #3: we specify the number of thread-blocks and the number of threads per thread-block to use through the arguments placed in the unconventional triple chevrons when we launch a compute kernel (the `<<< ... >>>` decoration in the `fillKernel` call).

Note #4: we have used three CUDA API calls here, namely `cudaMalloc`, `cudaMemcpy`, and `cudaFree`. Their names are fairly self evident and you can find out more details about them through your favorite search engine.

Note #5: the kernel launch is an asynchronous, non-blocking request. The launch request is sent to the GPU and the HOST program execution continues before the kernel is guaranteed to have finished.

Note #6: the `cudaMemcpy` function blocks until all previously queued kernels and the data copying requested have completed. Thus in this code after the `cudaMemcpy` returns we know that the kernel execution has completed and that the data has been copied to the HOST array.

⁷In the CUDA runtime API

8.9.2 CUDA hello world: DEVICE code

The HOST side code above makes a call to code that executes on the DEVICE. Understanding the syntax and specifics of the CUDA programming model for executing tasks on the GPU is one of the greatest hurdles for a new GPU programmer. Before we dive into the actual GPU kernel code let's consider a simple serial function that fills the entries in an array as follows

```
1 void fillSerial(int N, int val, int *h_a){
2
3     for(int n=0;n<N;++n){
4         h_a[n] = val;
5     }
6
7 }
```

Unfortunately we cannot use this code as is to program the GPU task. The most immediate problem is that the “for” loop is suitable for a serial processor to sequentially perform the array value assignments. We are going to mutate this code step by step until it is suitable for a GPU processor by adding one concept at a time.

First we recall that a GPU has an architecture with two levels of parallelism: multiple compute cores that each has multiple thread processors. Thus we are going to split the fill loop into two nested loops where we are contemplating parallelizing the outer loop by distributing outer iterations to cores and inner loop iterations to thread processors in each core.

```
1 void fillSerialTwoLevel(int B, int N, int val, int *h_a){
2
3     for(int b=0;b<(N+B-1)/B;++b){ // outer loop
4         for(int t=0;t<B;++t){ // inner loop
5             int n = t + b*B;
6             if(n<N){ // check n is in [0,N)
7                 h_a[n] = val;
8             }
9         }
10    }
11 }
```

This should be reminiscent of OpenMP a little bit where we split for loops amongst threads, although we have not added the syntactical sugar needed to make this a parallel code. This type of loop splitting is sometimes also called loop-chunking.

The next step in making this GPU worthy CUDA code might be the most confusing thing about programming GPUs. The double nested loop is now organized in exactly the form we need for CUDA. Each iteration of the outer loop can be processed by a GPU core, and each iteration of the inner loop for that outer iteration can be processed by a thread-processor in the SIMD vector unit for that core.

We can now mutate the `fillSerialTwoLevel` function into a CUDA kernel with the syntactical

changes shown in the code below where we have removed the explicit mention of the for loops that are not needed anymore since each iteration is handled by a thread.

```

1  __global__ void fillKernel(int N, int val, int *c_a){
2
3      // find index of thread relative to thread-block
4      int t = threadIdx.x;
5
6      // find index of thread-block relative to grid of thread-blocks
7      int b = blockIdx.x;
8
9      // find number of threads in thread-block
10     int B = blockDim.x;
11
12     // map from thread and thread-block indices into linear array index
13     int n = t + b*B;
14
15     // check index is in range
16     if(n<N)
17         c_a[n] = val; // work done by thread
18
19 }
```

There are several novel aspects to the kernel that we describe in the following notes.

Note #1: we tell the CUDA compiler that this is a CUDA DEVICE kernel with the annotation `__global__`.

Note #2: this DEVICE side code is executed by every thread in every block of threads launched on the DEVICE by the HOST side code.

Note #3: we should only pass DEVICE pointers into that third argument when invoking the kernel, since the code is executing on the DEVICE and only has efficient direct access to global DEVICE memory.

Note #4: we find the rank of the thread relative to its thread-block using the intrinsic `threadIdx` struct variable that is provided to each thread.

Note #5: we find which thread-block the current thread belongs to through the intrinsic `blockIdx` struct variable that is provided to each thread.

Note #6: we find out how many threads are in each thread-block through the intrinsic `blockDim` struct variable that is provided to each thread.

Note #7: once a thread knows its thread number, thread-block number, and the number of threads per thread-block, it uses these numbers to identify the index value for the array entry it will fill. Each thread in the above code will only set one value.

Note #8: there is more than one way to decide based on thread rank and thread block rank what each thread should do. Try experimenting with the way `t` and `b` are used to select array index `n`.

Note #9: the thread block dimension is determined by the launch parameters inside the triple chevrons at the kernel call side in the HOST side code.

Note #10: when we partition the for loop and construct the thread map we should carefully check that there is no loop carried dependence, order of iteration dependence, array write conflict. In fact we need to check all the same conditions that we checked when we made a loop parallel with the OpenMP `#pragma parallel for` loop construct. The same conditions apply since we need to guarantee that the threads will not step on each other as they execute the kernel code.

It can be intimidating to realize that we are responsible as CUDA developers to decide on how many thread-blocks, threads per thread-block are needed and how the work should map into this set of threads. However, the development of many DEVICE compute kernels follows this approach of first writing the serial code then chunking the loops in a way that makes sense for the two level parallel architecture of the GPU.

8.9.3 CUDA hello world: compiling code

Once we have written a CUDA program and saved it with the `.cu` file extension we will use the NVIDIA CUDA compiler `nvcc` to compile it into an executable as follows.

```
nvcc -o cudaHelloWorld cudaHelloWorld.cu
```

However, there is major obstacle to doing this: the CUDA compiler `nvcc` is not always installed on a system. It can be installed in Ubuntu as part of the CUDA SDK with

```
sudo apt-get install cuda
```

but there is another problem. It is quite likely that your VirtualBox does not hardware virtualize your CUDA enabled GPU, if you even have one on your laptop.

So in summary, you probably do not have the CUDA compilers installed on your laptop and even if you do have the compilers you probably cannot access your NVIDIA GPU even if you have one installed.

Do not despair ! We are using the ARC cascades cluster precisely because some of the compute nodes are equipped with NVIDIA GPUs and the CUDA compilers/drivers/libraries are installed and can be made available using the module system.

8.10 CUDA you do this ?

Now consider the ramifications of all the requirements CUDA puts on the programmer, as introductory level CUDA programming obliges one to:

1. Explicitly allocate data array(s) on DEVICE.

2. Copy data from HOST array to DEVICE array.
3. Decide how many threads to use to process task on DEVICE.
4. Write threaded code for the DEVICE task processed by all threads.
5. Copy data result from DEVICE array to HOST array.
6. Not confuse data stored on DEVICE and HOST.

This extra programming burden has been enough to put many programmers off adopting CUDA in their code projects. Maintaining coherency between data that resides on HOST and data that resides on DEVICE can be a significant challenge for large software projects. Many times a programmer does not even know if a pointer to an array refers to a HOST heap array or a DEVICE heap array. This can easily happen in large programming projects with significant code bases where operations are performed by a collection of inscrutable or impenetrable black box software libraries.

NVIDIA is aware of this issue and added some features to CUDA to make the memory management between two memory spaces (HOST and DEVICE) issue less irksome. In particular they introduce so-called “unified virtual memory” (UVM) which we discuss in the next section.

8.11 CUDA unified virtual memory

With the prevalence of 64-bit pointers it is now possible to encode physical location of a memory address into extra bits in the pointer. Using excess bits in the pointer it is possible to indicate/determine whether an array is located on HOST, DEVICE, or with managed memory possibly both.

A programmer can allocate a DEVICE array with the `cudaMallocManaged` CUDA API function which I believe also allocates a HOST array at the same time as the DEVICE array is allocated. The HOST code can access and read/write via the managed pointer to the HOST side copy of the array. When a managed array pointer is passed to a CUDA DEVICE kernel the OS will detect when blocks of data from the HOST side array must be copied to the DEVICE side version of the array and will automatically migrate the data from HOST to DEVICE.

Using UVM the above HOST code for the CUDA hello world simplifies to

```
1 // use Unified Virtual Memory in hello world
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include "cuda.h"
5
6 int main(int argc, char **argv){
7
8     int N = 1024;
9
10    // 1. allocate managed HOST/DEVICE array
11    int *u_a;
12    cudaMallocManaged(&u_a, N*sizeof(int));
13
```

```

14 // 2. launch DEVICE fill kernel
15 int T = 256; // number of threads per thread block
16 dim3 G( (N+T-1)/T ); // number of thread blocks to use
17 dim3 B(T);
18
19 int val = 999; // value to fill DEVICE array with
20 fillKernel <<< G,B >>> (N, val, u_a);
21
22 // 3. synchronize with GPU (block until fillKernel finishes)
23 cudaDeviceSynchronize();
24
25 // 4. print out values on HOST
26 for(int n=0;n<N;++n) printf("u_a[%d] = %d\n", n, u_a[n]);
27
28 // 5. free arrays
29 cudaFree(u_a);
30
31 return 0;
32 }

```

Notice how the code is a little more streamlined than the original hello world CUDA program. We do not need to explicitly copy data between HOST and DEVICE as this is now automated through the use of CUDA managed memory. However, note that we had to add a `cudaDeviceSynchronize` call before printing out values from the managed array on the HOST. This is vital to guarantee that the kernel has finished on the DEVICE. It is critical to do this before we access the DEVICE array (`u_a`) from the HOST. Just reading from that array on the HOST actually requires data be copied behind the scenes from the DEVICE array to the HOST array.

There is a reasonable tutorial on using GPU UVM for beginners [here](#). However, before you get too excited it is important to note that longtime practioners of GPGPU computing find that the automatic migration of data between HOST and DEVICE may incur considerable overhead and reductions in performance. The just in time nature of the managed memory migration may not be as efficient as manually moving large blocks of data with `cudaMemcpy`.

8.12 CUDA timers

A key aspect of analyzing the performance of OpenMP and MPI codes was accurately estimating the time a task takes. With accurate elapsed time estimates we were able to convincingly explain the strong scaling characteristics of a distributed array reduction operation performed using MPI for instance. Fortunately the CUDA API includes a mechanism for accurately estimating how long it takes to complete computational tasks on the DEVICE too.

Timing tasks on a CUDA DEVICE is complicated slightly by the asynchronous nature of the DEVICE. In order to time how long something takes on the DEVICE we first have to recognize that even though a kernel launch command is issued now, it actually may not execute until some time in the future. Is it fair to time a task on a DEVICE when it is busy performing the previous ten tasks we asked of it ?

Instead of asking for the DEVICE wall clock at a given instance we insert an event into the DEVICE

job queue before queuing the tasks we wish to time and after those tasks have been queued. Then using the built in CUDA event timers we kind find out afterwards what the actual elapsed time was as far as the DEVICE is concerned between the start and end timing events were queued. In practice this gives very accurate timings for DEVICE tasks.

To demonstrate how this is done we added CUDA event based timings to the CUDA hello world program we saw earlier as follows.

```

1  // 1. allocate HOST array
2  int *h_a = (int*) calloc(N, sizeof(int));
3  int *c_a;
4
5  // 2. allocate DEVICE array
6  cudaMalloc(&c_a, N*sizeof(int));
7
8  // 3. create events
9  cudaEvent_t start, end;
10 cudaEventCreate(&start);
11 cudaEventCreate(&end);
12
13 // 4. calculate number of thread-blocks and threads per thread-block to use
14 int T = 256;
15 dim3 G( (N+T-1)/T );
16 dim3 B(T);
17
18 // 5. record start event
19 cudaEventRecord(start);
20
21 // 6. launch CUDA DEVICE kernel
22 int val = 999;
23 fillKernel <<< G,B >>> (N, val, c_a);
24
25 // 7. insert end record event in stream
26 cudaEventRecord(end);
27
28 // 8. copy data from DEVICE array to HOST array
29 cudaMemcpy(h_a, c_a, N*sizeof(int), cudaMemcpyDeviceToHost);
30
31 // 9. print out elapsed time
32 float elapsed;
33 cudaEventSynchronize(end);
34 cudaEventElapsedTime(&elapsed, start, end);
35 elapsed /= 1000.; // convert to seconds
36
37 printf("elapsed time: %g\n", elapsed);

```

In the above example we created two CUDA event objects **start** and **end** using `cudaEventCreate`. We then inject the **start** event into the job queue on the DEVICE before launching the kernel. Next we inject the **end** event into the DEVICE queue after the kernel. We guarantee that the timing events have completed by synchronizing HOST and DEVICE through calling `cudaEventSynchronize`. Finally we measure the time that elapsed in the processing of the DEVICE queue between the **start** and **end**

events.

For more details on using CUDA event based timings see [the NVIDIA CUDA developer blog](#).

8.13 CUDA: vector operations

In this section we give some examples of CUDA kernel implementations of array operations.

8.13.1 CUDA vector addition

The model we followed above in the `fillKernel` is typical of a CUDA kernel. The basic plan is to create a map between the thread index and thread-block index and a linear index. Each thread then uses this single index to decide which array entry or entries to work on.

In the following `addVectorsKernel` each thread computes the sum of a single entry from two arrays of `float` values and write it to a third array [In MATLAB: `c = a + b`]

```

1  __global__ void addVectorsKernel(int N, float *c_a, float *c_b, float *c_c){
2
3      // find index of thread relative to thread-block
4      int t = threadIdx.x;
5
6      // find index of thread-block relative to grid of thread-blocks
7      int b = blockIdx.x;
8
9      // find number of threads in thread-block
10     int B = blockDim.x;
11
12     // map from thread and thread-block indices into linear array index
13     int n = t + b*B;
14
15     // check index is in range
16     if(n<N)
17         c_c[n] = c_a[n] + c_b[n]; // work done by each thread
18
19 }
```

8.13.2 CUDA vector entry-wise multiplication

In the following `dotMultiplpyVectorsKernel` each thread computes entry-wise product of two arrays of `float` values and write it to a third array. [In MATLAB: `c = a.*b`]

```

1  __global__ void dotMultiplyVectorsKernel(int N, float *c_a, float *c_b, float *c_c){
2
3      // find index of thread relative to thread-block
4      int t = threadIdx.x;
5
6      // find index of thread-block relative to grid of thread-blocks
7      int b = blockIdx.x;
8
9      // find number of threads in thread-block
10     int B = blockDim.x;
11
12     // map from thread and thread-block indices into linear array index
13     int n = t + b*B;
14
15     // check index is in range
16     if(n<N)
17         c_c[n] = c_a[n] * c_b[n]; // work done by each thread
18
19 }

```

8.14 CUDA: summary of API functions

Summary of CUDA API functions as discussed above.

Action	CUDA API call
Allocate array on DEVICE	<code>cudaMalloc(&[POINTER], [NUMBER OF BYTES]);</code>
Allocate managed array on HOST/DEVICE	<code>cudaMallocManaged(&[POINTER], [NUMBER OF BYTES]);</code>
Copy data from HOST to DEVICE	<code>cudaMemcpy([DEVICE POINTER], [HOST POINTER], [NUMBER OF BYTES], cudaMemcpyHostToDevice);</code>
Copy data from DEVICE to HOST	<code>cudaMemcpy([HOST POINTER], [DEVICE POINTER], [NUMBER OF BYTES], cudaMemcpyDeviceToHost);</code>
Free DEVICE array	<code>cudaFree([DEVICE POINTER]);</code>
Launch kernel on DEVICE	<code>[KERNEL NAME] <<< [NUMBER OF THREAD BLOCKS], [THREADS PER THREAD-BLOCK] >>> ([ARGS]);</code>
Block until DEVICE queue flushes	<code>cudaDeviceSynchronize();</code>

Table 8.3: Summary of commonly used OpenMP API functions.

8.15 GPU DEVICE memory hierarchy

In this section, we give a brief overview of a contemporary big iron GPU memory hierarchy. We explain how the memory hierarchy is relevant to reduction type operations and develop a sequence

of CUDA reduction kernels to illustrate the impact of thread configuration, parallel partition, and memory access pattern need to be combined to achieve high performance.

There are research teams around the world who reverse engineer the hardware characteristics of commercially available GPUs using microbenchmarking. The teams develop CUDA compute kernels that can be used to profile the hardware performance and then run these kernels with a range of parameters to estimate critical performance metrics that are not otherwise made public by the GPU manufacturers. For example Table 3.1 in [1] reveals a great deal of information about the NVIDIA V100 GPU. The information in that table includes the size of the on memory caches in the DEVICE processor.

Location	Pool	Typical Size	Shared by	Purpose
HOST	System	up to 1TB	All HOST threads	CPU heap storage
DEVICE	System	16-32GB	All DEVICE threads	GPU global heap storage
DEVICE chip	L2	6MB		GPU global cache
	Unified cache	128KB/core	Threads in thread-block	GPU cache local to core
	Registers	4×64KB/core	Local to thread	GPU stack storage on each core

Table 8.4: Brief summary of different levels of CPU and GPU nonunified memory access (NUMA).

In Table 8.4 we summarize the different levels of the GPU nonuniform memory access (NUMA) architecture of the contemporary NVIDIA V100 GPU that we have access to on Cascades. There are separate system memory spaces that are physically attached to the HOST and DEVICE processors where we typically store heap data. We allocate an array on the HOST heap with standard library `malloc` function and on the DEVICE heap with `cudaMalloc` CUDA API call. In CUDA we use the same pointer variables to represent the starting address of DEVICE system memory arrays and the start of HOST memory arrays but we need to be very careful not to use DEVICE pointers to access HOST memory and vice versa⁸ In the last chapter we even used an informal naming convention where HOST pointers were named `h_pointerName` and DEVICE pointers were named `c_pointerName` precisely for this reason.

The physical DEVICE system memory is stored in 16 stacks of second generation high-bandwidth memory (HBM2) which is located in proximity to the DEVICE processor (see this [datasheet](#)). Despite the close proximity of the HBM2 stacks to the DEVICE processor the physical limits on signal transmission and finite logic switching times means that a GPU core request to read data from DEVICE memory may take 375 GPU clock cycles (see Figure 3.2 of [1]).

With earlier generation GPUs the programmer was expected to compensate for high latency DEVICE memory accesses with careful overlap of data movement and computation combined with oversubscribing each core with more threads than thread processors. When I first started GPU programming it was common for CUDA codes to run very slowly if they weren't carefully engineered to hide DEVICE memory latency.

Each subsequent GPU upgrade the memory system has become easier to work with, making it easier for GPU programmers to develop performant code. The NVIDIA V100 GPU for instance has a shared 6MB L2 cache that sits between the DEVICE system memory and the L1 caches associated with each GPU core. A data request from a DEVICE core first is considered by the L1 cache, if the requested data is not already loaded in the L1 cache the request is passed to the L2 cache, and if the data is not there then the request is passed through to the DEVICE system memory. If the requested data is found in one of the caches then the latency is reduced compared to if the request has to go all the

⁸In my view using vanilla pointers to represent DEVICE memory locations is one of the biggest missteps in CUDA as it leads to many programming mistakes when HOST and DEVICE pointers are confused.

way to the DEVICE system memory. The L1 cache has a hit latency of about 28 clock cycles, and the L2 cache has a hit latency of about 193 clock cycles. Thus if the data is cached it can reduce the time to read the data from 375 cycles to 193 cycles for an L2 hit or even 28 cycles for an L1 hit. Clearly an L1 hit is preferred, but we have to temper our expectations since each core only has 4 pools of 64KB storage in L1. Furthermore we have limited control over what data is retained in the L1 and L2 caches.

Fortunately, the unified L1 cache associated with each GPU core is actually split into an L1 cache (which we don't control closely) and a pool of shared memory accessible by all threads on the core (which we do control closely). In a CUDA kernel we can declare a stack array that is shared by all threads in a thread-block using the `__shared__` annotation. A shared array can then be used as a common scratch pad for all the threads in a thread-block, and is a much faster way to communicate data between threads than using the global DEVICE system memory and is more reliable than L1 since we explicitly manage which data is stored in the shared memory space.

8.16 CUDA vector reduction

In some regards the CUDA programming model and GPU architecture is very similar to programming CPUs with OpenMP. Both are built on a shared memory model with all threads able to access GPU system memory with CUDA or CPU system memory with OpenMP. In the following we show seven different implementations for performing a reduction operation on an array of doubles located in the shared DEVICE system memory. After discussing each of the implementations we give a summary of their performance in Section 8.16.8.

8.16.1 Kernel V1: single thread

We start really basic and implement the sum reduction

$$s = \sum_{m=0}^N a_m,$$

assuming only one thread and one thread-block. The following gives the most basic implementation in a CUDA kernel one could think of.

```
1 // sum reduction using one thread on GPU
2 __global__ void reductionKernelV1(int N, double *c_a, double *c_suma){
3
4     c_suma[0] = 0;
5     for(int m=0;m<N;++m)
6         c_suma[0] += c_a[m];
7
8 }
```

The only thing that gives this away as CUDA code is that the kernel is annotated with `__global__`. We do not expect this code to perform very well as the single thread will stall for about 400 clock cycles every time it requests data from global DEVICE memory.

Note: this is a really naïve implementation as we keep incrementing a memory location in global DEVICE memory.

We created a HOST function for each of the seven kernels that specifies the thread grid and uses CUDA event timing to time how long the kernel takes to execute. We include just the V1 launch and timing function here for illustrative purposes.

```

1 void timeKernelV1(int N, double *c_a, double *c_suma, double *h_suma, double *elapsed){
2
3     // 1. create events
4     cudaEvent_t start, end;
5     cudaEventCreate(&start);
6     cudaEventCreate(&end);
7
8     // 2. record start event
9     cudaEventRecord(start);
10
11    // 3. launch CUDA DEVICE kernel with one thread in one thread-block
12    reductionKernelV1 <<< 1,1 >>> (N, c_a, c_suma);
13
14    // 4. insert end record event in stream
15    cudaEventRecord(end);
16
17    // 8. copy data from DEVICE array to HOST array
18    cudaMemcpy(h_suma, c_suma, 1*sizeof(double), cudaMemcpyDeviceToHost);
19
20    // 9. print out elapsed time
21    float felapsed;
22    cudaEventSynchronize(end);
23    cudaEventElapsedTime(&felapsed, start, end);
24    *elapsed = felapsed/1000.; // convert to seconds
25
26 }
```

For subsequent kernel launchers we specify sufficient threads with the thread grid as described in the description of the kernel.

8.16.2 Kernel V2: single thread with local accumulator

The V1 reduction code unnecessarily kept incrementing a global DEVICE memory location which will incur extra latencies even if that variable is cached in L1 memory. We rectify that in the V2 kernel by using a local stack variable for the single thread.

```

1 // one thread with local stack variable for accumulator
2 __global__ void reductionKernelV2(int N, double *c_a, double *c_suma){
3
4     double res = 0;
```

```

5
6   for(int m=0;m<N;++m)
7       res += c_a[m];
8
9   c_suma[0] = res;
10  }

```

This may improve the overall performance by reducing cache accesses, but still cannot exercise more than a fraction of the overall memory bandwidth of the GPU. By requesting one double (8 bytes, so 64 bits) at a time we are chronically under utilizing the wide memory bus which on the NVIDIA V100 is 4096 bits wide. This combined with only using one thread on one core with no oversubscription of the GPU cores will make this kernel still very slow.

8.16.3 Kernel V3: N threads but with race conditions

To increase the number of threads we can choose to use N threads and only for demonstration purposes we show what happens when they all try to accumulate to a single accumulation variable.

```

1  // BAD CODE: reduction kernel with N threads, but prone to read-write race conflicts
2  __global__ void reductionKernelV3(int N, double *c_a, double *c_suma){
3
4      int t = threadIdx.x;
5      int b = blockDim.x;
6      int B = blockDim.x;
7
8      int n = t + b*B;
9      if(n<N){
10         // dangerous: will cause read-write race conflicts
11         c_suma[0] += c_a[n];
12     }
13 }

```

This kernel will actually get the wrong answer when multiple threads read the accumulator, increment it, and then overwrite the increments from other threads that happened between read and write.

8.16.4 Kernel V4: N threads using uninterruptible atomic adds

One way to fix the read-write race conditions that arise in kernel V3 is to use the protected CUDA `atomicAdd` function. This will guarantee that incrementing of the accumulator will happen without being interrupted by other threads.

```

1  __global__ void reductionKernelV4(int N, double *c_a, double *c_suma){
2
3      int t = threadIdx.x;

```

```

4   int b = blockIdx.x;
5   int B = blockDim.x;
6
7   int n = t + b*B;
8   if(n<N){
9       double an = c_a[n];
10      // an uninterruptible increment
11      atomicAdd(c_suma, an);
12  }
13 }

```

This kernel will be faster than the single thread variants but will be slower than the broken V3 kernel. It is easy to see why, because the `atomicAdd` operations although efficiently handled in the L1/L2 caches will still require serialization of the increments to guarantee that they will all be accounted for.

The main bottleneck for the V4 kernel is the collision reconciliation that happens when N threads all try to increment a single variable. One way to improve the performance of this code is to use multiple accumulators, we leave this to the reader as an exercise.

8.16.5 Kernel V5: shared memory based thread-block based tree reduction

In kernel V4 we introduced one new aspect of CUDA kernel programming: an atomic addition operation that allows us to avoid read-write race conflicts when incrementing an accumulator in global DEVICE memory. The atomic operation is performed within the L1, L2, or global DEVICE memory. The latencies of atomic operations in either shared or global memory are given in Table 4.2 of [1]. The latency for an atomic operation range from 6 cycles when done in shared memory with no concurrency conflict to 36 cycles when done in global DEVICE memory with no conflict. The latency increase in both cases with increased contention, i.e. when more threads try to concurrently perform the atomic operation. Unfortunately the atomic operation has a significant potential overhead and in the above atomic based reduction the probability of contention between many threads is very high.

We can reduce the probability of contention between many threads when atomically incrementing a shared variable in several ways. One way is to use multiple accumulating variables in a similar manner that we used in OpenMP, for instance by creating an accumulator for each thread-block or reducing the number of threads that try to accumulate to a global DEVICE memory. A classic way to go about this is to perform a thread-block-wise reduction using a shared memory array for each thread-block. We will adapt the tree-wise reduction operation to coordinate a thread-block based partial reduction.

Each thread-block can include up to 1024 threads in the contemporary version of CUDA. We can start the reduction by tasking each thread with loading a single value from DEVICE global memory and then storing it in a shared memory array. In the next stage we idle half of the threads, and the remaining 512 threads add two values from shared and store one value back to shared. We can then repeat this process of reducing the number of active threads and adding in a further 8 rounds until there is only one active thread that we then nominate to increment the global DEVICE accumulator using an atomic addition operation. Doing this we will significantly reduce the instances of contention in incrementing the global accumulator.

There are two new CUDA kernel language options we need to implement this. The first is to declare

a stack array within the kernel using the `__local__` annotation. This lets the CUDA compiler know that the stack array should be mapped into the unified L1/shared memory cache on the core and be made accessible to all threads in the thread-block. The next thing we need to do is make sure that every time any thread has written to the shared memory array then we introduce a synchronization barrier (also known as a memory fence) before any thread will read from that part of shared memory. The synchronization is important to flush all writes to the shared array before further threads read from the shared array, otherwise it is not guaranteed that the writes will proceed the reads.

There is an excellent discussion of how to optimize shared memory based reductions in [this presentation](#) by Mark Harris of NVIDIA. However, his more aggressive optimizations for the reduction involved intra-SIMD synchronization. That is he relied on the 32-wide SIMD units concurrently executing instructions so that he could avoid using barriers during the last five binary tree reduction steps. Even then his optimized code neglected to declare that the shared memory be declared as “volatile” to force the compiler not to optimize away writes to the shared array. Finally, even if the shared array is declared as volatile in the latest GPU core architectures SIMD concurrency is no longer guaranteed. Thus several of his optimizations are considered brittle, although his reduction kernel implementations can be brought up to date by introducing calls to warp synchronization commands.

In the following we provide a simplified thread-block reduction kernel to illustrate how to use shared memory collaboratively between threads in a thread-block and how to synchronize between writes and reads to the shared memory array.

```

1  // use power of 2 number of threads per thread-block
2  #define p_B 1024
3  __global__ void reductionKernelV5(int N, double *c_a, double *c_suma){
4
5      __shared__ double s_a[p_B];
6
7      int t = threadIdx.x;
8      int b = blockIdx.x;
9      int B = blockDim.x;
10
11     int n = t + b*B;
12     double an = (n<N) ? c_a[n]:0;
13
14     s_a[t] = an;
15
16     int alive = p_B/2;
17     while(alive>0){
18
19         __syncthreads();
20
21         if(t<alive)
22             s_a[t] += s_a[t+alive];
23         alive /= 2;
24     }
25
26     if(t==0){
27         double an = s_a[0];
28         // an uninterruptible increment
29         atomicAdd(c_suma, an);
30     }

```

```
31 }
```

Note: in order to use the shared memory as a stack array in this code the compiler needs to know the size of the array at compilation time, and thus we used a preprocessor variable (`p_B`) set to 1024 as the size for the array. We could have done some more artful things like for instance declaring this kernel as a C++ style templated kernel with say `p_B` provided as a templated parameter.

```
1 // use power of 2 number of threads per thread-block
2 template <p_B>
3 __global__ void reductionKernelV5(int N, float *c_a, float *c_suma){
4
5     ...;
6 }
7
8 ...
9
10 templatedSharedtomicVectorReductionKernel<1024> <<< G,B >>> (N, c_a, c_suma);
```

8.16.6 Kernel V6: unrolling and warp based synchronization

In kernel V5 we deployed a manually managed shared memory cache as a local scratch pad so that the threads in a thread-block could collaboratively reduce 1024 array values to one. This required ten iterations of a tree reduction process. Unfortunately this meant that we need to introduce ten thread-block synchronizations in order to guarantee that the read and writes for each round of the tree reduction were completed by all threads before the next round. In kernel V6 we refine this strategy and group the 1024 threads into 32 groups of 32 threads to match the 32 wide SIMD width of the GPU core. We proceed as before but this time we do two rounds of 32 way reductions, first involving all 32 groups then in the second round only involving 32 threads in a single group. This allows us to replace the expensive `__syncthreads()` calls which synchronize all the threads in the thread-block with calls to `__syncwarp()` which only synchronizes threads in each “warp”, i.e. in each group of 32 threads passing through the SIMD units collectively.

In order to perform the warp synchronized block reduction efficiently we also unroll the `while` loop, which in itself improves efficiency by removing some conditional branching. Finally we use a two-dimensional thread configuration of 32×32 threads as this allows us to identify which warp a thread is in easily by using both `threadIdx.x` and `threadIdx.y`. The code for kernel V6 follows.

```
1 // (32 threads in each SIMD group are synchronized via syncwarp)
2 #define p_W 32
3
4 __global__ void reductionKernelV6(int N, double *c_a, double *c_suma){
5
6     __shared__ double s_a[p_W][p_W];
7 }
```

```

8  int tx = threadIdx.x, ty = threadIdx.y;
9  int b = blockIdx.x;
10 int BX = blockDim.x, BY = blockDim.y;
11
12 int n = tx + ty*BX + b*BX*BY;
13 double an = (n<N) ? c_a[n]:0;
14
15 // initial load
16 s_a[ty][tx] = an;
17 __syncwarp();
18
19 // first binary tree reduction
20 {
21     if(tx<16) s_a[ty][tx] += s_a[ty][tx+16];
22     __syncwarp();
23
24     if(tx< 8) s_a[ty][tx] += s_a[ty][tx+8];
25     __syncwarp();
26
27     if(tx< 4) s_a[ty][tx] += s_a[ty][tx+4];
28     __syncwarp();
29
30     if(tx< 2) s_a[ty][tx] += s_a[ty][tx+2];
31     __syncwarp();
32
33     // special choice to avoid bank conflicts
34     if(tx< 1) s_a[ty][ty] = s_a[ty][0] + s_a[ty][1];
35 }
36
37 // synchronize all warps
38 __syncthreads();
39
40 // second binary tree reduction using 0 warp
41 if(ty==0){
42     if( tx<16) s_a[0][tx] = s_a[tx][tx] + s_a[tx+16][tx+16];
43     __syncwarp();
44
45     if( tx< 8) s_a[0][tx] += s_a[0][tx+8];
46     __syncwarp();
47
48     if( tx< 4) s_a[0][tx] += s_a[0][tx+4];
49     __syncwarp();
50
51     if( tx< 2) s_a[0][tx] += s_a[0][tx+2];
52     __syncwarp();
53
54     if( tx< 1){
55         double res = s_a[0][0] + s_a[0][1];
56         // an uninterruptible increment
57         atomicAdd(c_suma, res);
58     }
59 }
60 }

```

This may seem complicated but each step is carefully considered to minimize the number of expensive synchronizations.

8.16.7 Kernel V7: multiple data loads per thread

The V6 kernel is in relatively good shape. It exploits the shared memory cache to reduce data movement, guarantees thread safety by exploiting atomic adds to increment the accumulator, reduces thread synchronization, unrolls an expensive `while` loop, with a couple of other performance sweeteners that are a little beyond the scope of this course. However, the block reduction is still relatively expensive incurring 9 warp synchronizations and a thread-block synchronization on top of several accesses to shared memory.

We reduce the number of times the thread-block reduction has to happen following Mark Harris' reduction optimization plan [3]. The final step in the optimization is to reduce the number of thread-blocks by making each thread load more than one array value. This trades parallelism for lower operation count.

The kernel code for V7 is the same for V6 except for the initial data reading as follows

```

1 // reduce number of tree reductions - each thread loads multiple values
2 __global__ void reductionKernelV7(int N, double *c_a, double *c_suma){
3
4     __shared__ double s_a[p_W][p_W];
5
6     int tx = threadIdx.x;
7     int ty = threadIdx.y;
8     int b = blockIdx.x;
9     int BX = blockDim.x;
10    int BY = blockDim.y;
11    int Nthreads = BX*BY*gridDim.x;
12
13    int n = tx + ty*BX + b*BX*BY;
14    double an = 0;
15
16    while(n<N){
17        an += c_a[n];
18        n += Nthreads;
19    }
20
21    // initial load
22    s_a[ty][tx] = an;
23    __syncwarp();
24    ....; // same as V6
25 }
```

We then choose the number of thread-blocks so that the kernel loads say 2,3, or 4 values and perform some experiments to determine which yields the maximum throughput.

8.16.8 Reduction kernels: performance results

In the proceeding sections we have described a sequence of deliberate optimizations to improve the performance of a baseline naïve reduction implementation. The results of running these different variants on an NVIDIA Titan V are shown in Table 8.5. We see as expected that the V1 kernel achieves a tiny fraction of possible data throughput. Given that the NVIDIA Titan V has a [manufacturer peak bandwidth of 652 GB/s](#) it is remarkable how poor the performance of the single thread V1 and V2 kernels is, even if the V2 does eek out a little performance improvement over V1.

The unprotected increments of the V3 kernel yield an incorrect result for the reduction as seen in the last column of Table 8.5 which should be 20 million not 196.

The atomic variant V4 does a very credible job achieving more than 50% of the peak throughput. However, the results are much more impressive for the shared memory caching variants V5-V7. We see that the V6 and V7 variants in particular achieve more than 90% of peak memory efficiency. This is remarkable given that the reduction process is inherently serializing, yet by careful kernel design we have been able to almost fully exploit the GPU memory bandwidth.

Kernel V#	Notes	Elapsed (s)	Loads (GB/s)	Result
1	Serial	1.78e+00	9.00e-02	20000000
2	Improved serial	6.19e-01	2.59e-01	20000000
3	Read/write conflicts	1.03e-03	1.55e+02	196
4	All atomic adds	4.74e-02	3.38e+00	20000000
5	Shared tree & atomic	3.13e-04	5.11e+02	20000000
6	V5 + reduced sync & atomic	2.64e-04	6.06e+02	20000000
7	V6 + multi-loads per thread	2.62e-04	6.10e+02	20000000

Table 8.5: CUDA sum reductions for 7 kernel implementations. Experiments conducted on an NVIDIA Titan V consumer GPU, involving reducing an array of 20 million doubles, each double set to 1.

We repeated the experiment on the ARC Cascades system. The results for this experiment are shown in Table 8.6. Whereas the consumer grade NVIDIA Titan V has a manufacturer spec of 652 GB/s, the NVIDIA V100 GPU has more HBM2 memory modules and has a [claimed theoretical peak bandwidth of 900GB/s](#).

Kernel V#	Notes	Elapsed (s)	Loads (GB/s)	Result
1	Serial	1.74e+00	9.20e-02	20000000
2	Improved serial	6.12e-01	2.62e-01	20000000
3	Read/write conflicts	9.99e-04	1.60e+02	208
4	All atomic adds	4.57e-02	3.50e+00	20000000
5	Shared tree & atomic	2.98e-04	5.37e+02	20000000
6	V5 + reduced sync & atomic	2.27e-04	7.04e+02	20000000
7	V6 + multi-loads per thread	1.90e-04	8.41e+02	20000000

Table 8.6: CUDA sum reductions for 7 kernel implementations. Experiments conducted on an NVIDIA V100 server grade GPU on cascades, involving reducing an array of 20 million doubles, each double set to 1.

The V100 results have a similar trend to the Titan V, however the V7 kernel optimizations reducing the number of thread-blocks yield a larger gain over V6. The V7 kernel achieves a data load throughput of 841 GB/s. This is a phenomenal 93% of peak performance ! Thus there really is almost no penalty in performing a reduction on a GPU as it can be done at the same throughput as reading from the

array.

References

- [1] Zhe Jia et al. “Dissecting the Nvidia volta GPU architecture via microbenchmarking”. In: *arXiv preprint arXiv:1804.06826* (2018).
- [2] Ian Masliah et al. “High-performance matrix-matrix multiplications of very small matrices”. In: *European Conference on Parallel Processing*. Springer. 2016, pp. 659–671.
- [3] Stefano Markidis et al. “Nvidia tensor core programmability, performance & precision”. In: *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE. 2018, pp. 522–531.
- [4] Azzam Haidar et al. “Harnessing GPU tensor cores for fast FP16 arithmetic to speed up mixed-precision iterative refinement solvers”. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*. IEEE Press. 2018, p. 47.
- [5] Azzam Haidar et al. “The design of fast and energy-efficient linear solvers: On the potential of half-precision arithmetic and iterative refinement techniques”. In: *International Conference on Computational Science*. Springer. 2018, pp. 586–600.
- [6] Pierre Blanchard et al. “Mixed Precision Block Fused Multiply-Add: Error Analysis and Application to GPU Tensor Cores”. In: (2019).
- [7] Jason Sanders and Edward Kandrot. *CUDA by example: an introduction to general-purpose GPU programming*. Addison-Wesley Professional, 2010.
- [8] Volodymyr Kindratenko. *Numerical computations with GPUs*. Springer, 2014.
- [9] Rob Farber. *CUDA application design and development*. Elsevier, 2011.
- [10] John Cheng, Max Grossman, and Ty McKercher. *Professional Cuda C Programming*. John Wiley & Sons, 2014.
- [11] David B Kirk and W Hwu Wen-Mei. *Programming massively parallel processors: a hands-on approach*. Morgan kaufmann, 2016.
- [12] Benedict Gaster et al. *Heterogeneous computing with openCL: revised openCL 1*. Newnes, 2012.